

---

## Endpoints disponibles

---

### Sistema

#### GET /

Verifica que la API esta activa.

Respuesta:

```
{
  "message": "Nodo Civico API activa",
  "status": "ok"
}
```

#### GET /health

Devuelve el estado actual de la API y su version.

Respuesta:

```
{
  "status": "ok",
  "version": "2.0.0"
}
```

---

### Usuarios

#### POST /users/register

Registra un nuevo usuario. Si el correo ya existe, devuelve el usuario existente.

Cuerpo de la peticion:

```
{
  "name": "Carlos Perez",
  "email": "carlos@ejemplo.com",
  "sector": "Barrio Norte",
  "password": "opcional1234"
}
```

Campos obligatorios: `name`, `email`.

Respuesta exitosa (201 o 200 si ya existe):

```
{
  "id": 1,
  "name": "Carlos Perez",
  "email": "carlos@ejemplo.com",
  "sector": "Barrio Norte",
  "created_at": "2026-05-22T19:00:00"
}
```

#### POST /users/login

Identifica a un usuario por correo electronico.

Cuerpo de la peticion:

```
{
  "email": "carlos@ejemplo.com"
}
```

Respuesta exitosa (200):

```
{
  "id": 1,
  "name": "Carlos Perez",
  "email": "carlos@ejemplo.com",
  "sector": "Barrio Norte",
  "created_at": "2026-05-22T19:00:00"
}
```

Respuesta cuando no existe (404):

```
{
  "error": "user_not_found"
}
```

### GET /users/{id}

Devuelve los datos de un usuario por su identificador numerico.

Respuesta exitosa (200): misma estructura que `/users/login`.

### GET /users

Devuelve la lista de todos los usuarios registrados.

Respuesta exitosa (200):

```
[
  {
    "id": 1,
    "name": "Carlos Perez",
    "email": "carlos@ejemplo.com",
    "sector": "Barrio Norte",
    "created_at": "2026-05-22T19:00:00"
  }
]
```

---

## Categorias

### GET /categories

Devuelve la lista de categorias disponibles para clasificar los reportes.

Respuesta exitosa (200):

```
[
  { "id": 1, "name": "Alumbrado", "iconEmoji": "" },
  { "id": 2, "name": "Aseo", "iconEmoji": "" },
  { "id": 3, "name": "Seguridad", "iconEmoji": "" },
  { "id": 4, "name": "Servicios publicos", "iconEmoji": "" }
]
```

### POST /categories

Crea una nueva categoria.

Cuerpo de la peticion:

```
{
  "name": "Infraestructura",
  "iconEmoji": ""
}
```

Campo obligatorio: `name`.

Respuesta exitosa (201):

```
{
  "id": 5,
  "name": "Infraestructura",
  "iconEmoji": ""
}
```

## Reportes

Todos los endpoints de reportes son publicos y no requieren cabecera de autorizacion.

### GET /reports

Devuelve todos los reportes ordenados por fecha de creacion descendente.

Respuesta exitosa (200):

```
[
  {
    "id": 1,
    "remoteId": "1",
    "userId": 1,
    "title": "Luminaria danada en la calle 12",
    "description": "La lampara lleva 3 noches apagada.",
    "category": "Alumbrado",
    "priority": "ALTA",
    "status": "OPEN",
    "location": "Calle 12 con carrera 5",
    "pendingSync": false,
    "createdAt": 1748000000000,
    "updatedAt": 1748000000000
  }
]
```

### GET /reports/{id}

Devuelve el detalle de un reporte por su identificador.

Respuesta exitosa (200): estructura identica a un item del listado.

Respuesta cuando no existe (404):

```
{
  "error": "Not Found"
}
```

### GET /reports/all

Alias de `GET /reports`. Devuelve todos los reportes sin filtro.

### POST /reports

Crea un nuevo reporte.

Cuerpo de la peticion:

```
{
  "title": "Hueco en la via principal",
  "description": "Hueco de aproximadamente 40 cm en el carril derecho.",
  "category": "Infraestructura",
  "priority": "ALTA",
  "status": "OPEN",
  "location": "Carrera 10 con calle 5",
  "userId": 1
}
```

Campos obligatorios: title, description, category, priority .

Valores validos para priority : LOW, MEDIA, ALTA .

Valores validos para status : OPEN, IN\_PROGRESS, RESOLVED, CLOSED .

Respuesta exitosa (201): estructura completa del reporte creado.

Respuesta con campos faltantes (400):

```
{
  "error": "missing_fields",
  "fields": ["title", "category"]
}
```

PUT /reports/{id}

Actualiza todos los campos editables de un reporte existente.

Cuerpo de la peticion (todos los campos son opcionales):

```
{
  "title": "Nuevo titulo",
  "description": "Descripcion actualizada.",
  "category": "Aseo",
  "priority": "MEDIA",
  "status": "IN_PROGRESS",
  "location": "Parque central"
}
```

Respuesta exitosa (200): estructura completa del reporte actualizado.

PATCH /reports/{id}/status

Actualiza unicamente el estado de un reporte.

Cuerpo de la peticion:

```
{
  "status": "RESOLVED"
}
```

Respuesta exitosa (200): estructura completa del reporte con el nuevo estado.

Respuesta sin campo status (400):

```
{
  "error": "status_required"
}
```

DELETE /reports/{id}

Elimina un reporte de forma permanente.

Respuesta exitosa (200):

```
{
  "deleted": true,
  "id": 3
}
```

Estructura del objeto Report

| Campo | Tipo    | Descripcion                                       |
|-------|---------|---------------------------------------------------|
| id    | Integer | Identificador unico generado por la base de datos |
|       |         |                                                   |

| remotedId<br>Campo | String<br>Tipo | Representacion en texto del id, para uso en Android<br>Descripcion |
|--------------------|----------------|--------------------------------------------------------------------|
| userId             | Integer        | Identificador del usuario que creo el reporte                      |
| title              | String         | Titulo corto del reporte                                           |
| description        | String         | Descripcion detallada                                              |
| category           | String         | Nombre de la categoria                                             |
| priority           | String         | Nivel de prioridad: LOW, MEDIA o ALTA                              |
| status             | String         | Estado actual: OPEN, IN_PROGRESS, RESOLVED o CLOSED                |
| location           | String         | Referencia geografica textual                                      |
| pendingSync        | Boolean        | Indica si el reporte tiene cambios pendientes de sincronizar       |
| createdAt          | Long           | Fecha de creacion en milisegundos (epoch UTC)                      |
| updatedAt          | Long           | Fecha de ultima actualizacion en milisegundos (epoch UTC)          |

---

## Codigos de respuesta HTTP

| Codigo | Significado                          |
|--------|--------------------------------------|
| 200    | Operacion exitosa                    |
| 201    | Recurso creado correctamente         |
| 400    | Peticion invalida o campos faltantes |
| 404    | Recurso no encontrado                |
| 500    | Error interno del servidor           |

---

## Configuracion en Android con Retrofit

En la capa de red de la aplicacion Android, la URL base debe apuntar al servidor donde corre la API. Ejemplo con ngrok:

```
private static final String BASE_URL = "https://abc123.ngrok-free.app/";

Retrofit retrofit = new Retrofit.Builder()
    .baseUrl(BASE_URL)
    .addConverterFactory(GsonConverterFactory.create())
    .build();
```

No se requiere ningun encabezado de autorizacion. Las peticiones se realizan directamente con el cuerpo JSON correspondiente a cada endpoint.

---